

1. Overview

This document outlines the final optimized architecture for the `base_query_info` pipeline in PostgreSQL, taking into account: - Large tables (`sql_statement_info` and `query_log`) - Incremental data loading per `version_id` - Optional joins with `table_metadata` - Partitioning for performance

The goal is to improve query performance, reduce unnecessary joins, and provide maintainable, scalable architecture.

2. Base Table (Partitioned by `version_id`)

```
CREATE TABLE base_query_info_core (  
    version_id INTEGER NOT NULL,  
    s_id BIGINT,  
    log_id BIGINT,  
    statement_type TEXT,  
    database TEXT,  
    schema TEXT,  
    query TEXT,  
    target_database TEXT,  
    target_schema TEXT,  
    target_entity_name TEXT,  
    source_database TEXT,  
    relationship_type TEXT  
) PARTITION BY LIST (version_id);
```

- Each `version_id` gets a separate partition for performance. - Queries filtering on `version_id` will scan only the relevant partition.

3. Creating Partitions

For each new version run:

```
CREATE TABLE base_query_info_v105 PARTITION OF base_query_info_core  
FOR VALUES IN (105);
```

- Partitions are inserted into once per ETL run. - Old partitions can be dropped efficiently.

4. Incremental Insert Query

```
INSERT INTO base_query_info_core
SELECT
    {{version}} AS version_id,
    s.id AS s_id,
    s.log_id,
    s.statement_type,
    s.database,
    s.schema,
    q.sql_query AS query,
    er.target_database,
    er.target_schema,
    er.target_entity_name,
    er.source_database,
    er.relationship_type
FROM sql_statement_info s
JOIN query_log q
    ON q.log_id = s.log_id AND q.version_id = s.version_id
JOIN entity_relationship er
    ON s.id = er.sql_statement_info_id
JOIN statement_context sc
    ON s.id = sc.sql_statement_info_id
WHERE s.version_id = {{version}};
```

- `sql_query` is fetched once and stored permanently. - No repeated join with `query_log` is needed afterward.

5. Indexing Strategy (per Partition)

```
CREATE INDEX idx_bqic_core_log_id ON base_query_info_v105(log_id);
CREATE INDEX idx_bqic_core_target_entity
    ON base_query_info_v105(target_database, target_schema, target_entity_name);
```

- Optional global index on `version_id` not required due to partition pruning.

6. Optional Metadata View

```
CREATE OR REPLACE VIEW base_query_info_with_metadata AS
SELECT
    c.*,
    COALESCE(tm.size_mb, 0.0) AS table_size
FROM base_query_info_core c
LEFT JOIN table_metadata tm
    ON tm.database = c.target_database
    AND tm.schema = c.target_schema
    AND tm.table_name = c.target_entity_name;
```

- Only scripts needing `table_size` will use this view. - Avoids unnecessary join cost for scripts that don't need metadata.

7. Optional Latest Version View

```
CREATE OR REPLACE VIEW base_query_info_latest AS
SELECT *
FROM base_query_info_core
WHERE version_id = (SELECT MAX(version_id) FROM base_query_info_core);
```

- Provides an easy way to query the newest version without specifying a version.

8. Maintenance Guidelines

Task	Command	Frequency	Notes
Vacuum & Analyze	<code>VACUUM ANALYZE base_query_info_core;</code>	Weekly	Keeps planner statistics up-to-date
Drop old versions	<code>DROP TABLE base_query_info_v{old};</code>	Monthly/ Quarterly	Efficient due to partitioning
Refresh metadata MV	<code>REFRESH MATERIALIZED VIEW CONCURRENTLY base_query_metadata_mv;</code>	Daily	Optional, if used
Automate partition creation	ETL Script	Each run	Ensures new version partitions exist before insert

9. Optional Metadata Cache

```
CREATE MATERIALIZED VIEW base_query_metadata_mv AS
SELECT
    database,
    schema,
    table_name,
    COALESCE(size_mb, 0.0) AS table_size
FROM table_metadata;
```

- Refresh periodically to speed up joins.

10. Key Benefits of This Design

- Single table stores all versions efficiently.
 - Partitioning ensures queries scan only relevant version data.
 - Indexes improve join and filter performance.
 - Optional metadata view avoids unnecessary expensive joins.
 - Easy maintenance: vacuuming, dropping old versions, refreshing metadata.
 - Supports incremental loads per version with no repeated joins.
 - Latest version view simplifies queries for scripts needing the newest data.
-

This design fully optimizes `base_query_info` for **large datasets, incremental versioned inserts, optional metadata, and maintainable performance** in PostgreSQL.